AI ML PROJECT REPORT

Piyush

31.03.2026

PROBLEM

The problem chosen was the 8 puzzle problem. Given a 3×3 board with 8 tiles (each numbered from 1 to 8) and one empty space, the objective is to place the numbers to match the final configuration using the empty space. We can slide four adjacent tiles (left, right, above, and below) into the empty space.

WHY THE PROBLEM MATTERS

Solving the 8-puzzle using Breadth-First Search (BFS) is significant because it provides a guaranteed optimal solution, ensuring the shortest path of moves is always found. In a project setting, this serves as a critical performance baseline to verify that more complex, "smarter" algorithms are actually reaching the most efficient goal. It also forces a practical engagement with memory management, as BFS must store every visited state to prevent infinite loops. By implementing this, a foundational understanding of exhaustive search and the computational trade-offs between absolute accuracy and system resources was demonstrated.

PERSONAL APPROACH TO SOLVING IT

The BFS algorithm was used to solve the 8 puzzle problem. Breadth-First Search (BFS) is a fundamental graph traversal algorithm used to explore nodes and edges systematically. It starts at a designated "root" node and inspects all neighboring nodes at the current depth level before moving on to the nodes at the next depth level.

KEY DECISIONS MADE, CHALLENGES FACED AND LESSONS LEARNED

A key decision in this project was implementing Breadth-First Search (BFS), specifically chosen because it guarantees the shortest path to the goal in an unweighted state space, ensuring the most efficient solution is always found. To support this, I opted for a Queue based approach paired with a hashed "visited set" to manage state tracking and prevent redundant processing.

The primary challenge I faced was the exponential growth of the state space; as the depth of the solution increased, the memory consumption rose sharply, illustrating the "state-space explosion" common in exhaustive searches. Additionally, handling unsolvable puzzle configurations proved tricky, requiring a pre-check of the inversion parity to avoid infinite loops.

Through this process, I learned the critical importance of data structure selection, specifically how using a set for O(1) lookups significantly optimizes performance, and I gained a practical understanding of the inherent trade-offs between algorithmic optimality and system resource constraints.